



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

VI SEMESTER B.Tech.
Data Sciences and Engineering
Principles of Artificial Intelligence Lab (DSE 3241)

Mini Project Report (April, 2026)

AI-Assisted Missing Person Search Simulation System

Submitted by:

Samarth Agrawal | Registration No.: 230968358

Team Number: Individual

Date of Submission: April 2026

1. Introduction

The identification and recovery of missing persons is one of the most resource-intensive and time-critical challenges faced by law enforcement agencies worldwide. Traditional investigation approaches are largely manual, reactive, and heavily reliant on human judgment under conditions of high uncertainty. Witness descriptions are frequently vague or qualitative, search areas are vast and unstructured, and the absence of computational decision-support tools often results in delayed response times and suboptimal resource allocation.

Artificial Intelligence (AI) offers a compelling framework to augment these processes. By combining search algorithms, fuzzy logic, and constraint satisfaction, it is possible to build a simulation system that interprets uncertain witness inputs, models the likely movement of a missing individual, and prioritises search zones within a 2-D environment — all while using entirely synthetic data to preserve ethical integrity.

This project presents the design and implementation of an AI-Assisted Missing Person Search Simulation System. The system accepts four witness-reported descriptors — height (in centimetres), age (in years), observed behaviour, and clothing colour — and processes them through a five-stage pipeline:

- Fuzzy Logic Inference (Mamdani + Centroid Defuzzification) to derive a crisp suspicion score from linguistic inputs.
- Constraint Satisfaction Problem (CSP) modelling to build a probability matrix over a grid environment, incorporating hotspot zone biases, time-decay from the last known location, behavioural patterns, and clothing-darkness biases.
- A 2-D Grid Environment simulating a city block, with configurable obstacle ratios and support for 4- or 8-directional movement.
- BFS and A* Search Algorithms to compute optimal paths from a fixed starting position to the most probable target location.
- Monte-Carlo Agent Simulation to model stochastic movement patterns of the missing individual and produce a density heatmap for visualisation.

The system is delivered both as a terminal application (`main.py`) and an interactive web interface built with Streamlit (`app.py`), making it accessible for demonstration and academic evaluation. All results are displayed through matplotlib visualisations, including fuzzy membership charts, aggregated output MFs, probability heatmaps, BFS vs. A* comparison grids, and Monte-Carlo density maps.

2. Literature Review

This section surveys the foundational and applied works that inform the algorithms and design choices in the present system.

2.1 Artificial Intelligence — Foundational Theory

Russell and Norvig's *Artificial Intelligence: A Modern Approach* (4th Edition, 2020) remains the definitive reference for intelligent agents, uninformed and informed search algorithms, constraint

satisfaction, and logical reasoning. The treatment of BFS as a complete, optimal uninformed search strategy and A* as the gold-standard informed search algorithm with an admissible heuristic directly underpins the search module (`search.py`) in this project. The text's discussion of CSP arc-consistency also motivates the hard-constraint gate (`is_valid_cell`) and the soft-constraint scoring engine (`ConstraintEngine`).

2.2 Fuzzy Logic for Linguistic Uncertainty

Lotfi Zadeh's seminal paper *Fuzzy Sets* (1965), introduced the concept of graded membership, enabling variables to belong to multiple linguistic categories simultaneously. The Mamdani inference method — popularised by Mamdani and Assilian (1975) — provides the rule-evaluation and aggregation framework used in `fuzzy.py`: antecedents are combined with the min T-norm, output membership functions are clipped at the firing strength, and the aggregated set is defuzzified via centroid (centre of gravity). This Mamdani pipeline is widely considered the most interpretable fuzzy inference architecture, making it ideal for explainable AI applications.

2.3 Probabilistic Search and Multi-Agent Systems

Stone (1975) and later Frost et al. (*IEEE Transactions on Aerospace and Electronic Systems*) formalised Bayesian search theory for missing-person scenarios, demonstrating that probability-of-detection (POD) maps derived from prior evidence can dramatically reduce the search area. The present project adopts a simplified version of this principle: the probability matrix (`generate_probability_grid` in `csp.py`) integrates zone-based hotspots, Gaussian distance-decay from the last-seen location, and behavioural bias — conceptually analogous to a POD map.

2.4 Reinforcement Learning and Adaptive Agents

Sutton and Barto's *Reinforcement Learning: An Introduction* (2018) provides the theoretical basis for adaptive agent behaviour in uncertain environments. Although the current project implements a fixed probabilistic walk rather than a trained policy, the `MissingPersonAgent` class in `search.py` draws directly from the concept of probability-weighted action selection, where the agent's movement is biased by a learned (here, analytically derived) value function represented by the probability matrix.

2.5 Search and Rescue Optimisation

Recent IEEE and ACM literature on search-and-rescue (SAR) robotics — including work by Queralta et al. (2020, *Frontiers in Robotics and AI*) — demonstrates that heuristic path planning (A*, D*-Lite) outperforms uninformed search in environments with structured prior information about target location. These findings motivated the use of a suspicion-weighted A* heuristic ($w = 1 + \text{suspicion_score}$, mapping the fuzzy output to an inflation factor), so that higher-confidence scenarios receive more aggressive greedy guidance toward the high-probability goal.

3. Observations

3.1 Drawbacks of Existing Approaches

Traditional and contemporary tools for missing person investigation exhibit several critical limitations:

- **Manual and Reactive Processing:** Current investigation workflows depend on officers manually sifting through witness statements without algorithmic filtering or prioritisation, resulting in inconsistent and slow decision-making.
- **Binary Handling of Linguistic Uncertainty:** Standard databases and alert systems require exact categorical inputs (e.g., exact height in centimetres, precise age). Witness reports are inherently imprecise; a description such as "tall and nervous" cannot be directly mapped to a database query.
- **Absence of Probabilistic Spatial Reasoning:** Existing tools do not integrate real-time spatial modelling. Search teams cover zones in a largely heuristic, experience-driven manner rather than following a probability-ranked map derived from witness evidence.
- **No Time-Decay Modelling:** As time elapses after a sighting, the individual's probable location shifts. Existing tools do not incorporate time-since-last-seen as a factor in spatial probability estimates.
- **Limited Algorithmic Comparison:** Investigations do not typically benchmark multiple search strategies against each other, making it difficult to quantify the efficiency gains from prioritised search.

3.2 How the Present Approach Addresses These Drawbacks

The system presented in this project directly addresses each of the above limitations:

- **Fuzzy Logic for Linguistic Inputs:** Instead of requiring exact values, the system accepts qualitative descriptors. Triangular and trapezoidal membership functions map crisp inputs to graded memberships across overlapping linguistic categories (e.g., a height of 175 cm receives membership 0.583 in "medium" and 0.278 in "tall"), capturing the inherent imprecision of witness descriptions.
- **Mamdani Inference for Explainability:** The rule base and centroid defuzzification produce a single, interpretable suspicion score with a transparent audit trail — operators can inspect every membership value and rule firing strength.
- **CSP-Driven Probability Maps:** The ConstraintEngine integrates hard constraints (obstacle blocking) and soft constraints (edge-proximity bias, quadrant bias, uniform prior) into a normalised probability matrix. This matrix guides both the biased goal placement and the Monte-Carlo agent simulation.
- **Time-Decay via Exponential Distance Penalty:** The `hours_since_seen` parameter exponentially reduces the probability mass near the last-known location, spreading probability to farther cells over time — a computationally lightweight proxy for diffusion-based search theory.
- **A* vs. BFS Benchmarking:** By running both algorithms on identical grids and reporting node-count and timing metrics, the system quantifies the efficiency gain from heuristic search — directly supporting the case for informed search in operational SAR contexts.

4. Methods Used

4.1 Fuzzy Logic Engine (fuzzy.py)

The fuzzy inference pipeline follows the Mamdani architecture with centroid defuzzification, implemented in `fuzzy.py`.

Fuzzification: Four input variables are fuzzified into overlapping linguistic sets using triangular (`trimf`) and trapezoidal (`trapmf`) membership functions:

- Height (cm): short [140–168], medium [155–182], tall [170–210].
- Age (years): young [15–32], adult [25–52], senior [45–80].
- Behaviour (discrete): calm, normal, nervous, very nervous — mapped directly to output-term memberships (low, medium, high).
- Clothing colour (discrete): eight colours mapped to output-term memberships; black carries the highest suspicion weight.

Rule Evaluation: A rule base of 26 weighted IF-THEN rules covers combinations of the four input variables. Each rule fires with $\text{strength} = \min(\mu_{\text{height}}, \mu_{\text{age}}, \mu_{\text{behavior}}, \mu_{\text{clothing}}) \times \text{rule_weight}$, implementing the min T-norm with soft weighting. The age antecedent may be set to None in a rule, in which case it is treated as 1.0 (the rule is age-independent).

Aggregation and Defuzzification: All clipped output MFs are aggregated by point-wise maximum (union). The centroid of the aggregated fuzzy set over the output universe [0.0, 1.0] (500 discrete points) yields the crisp suspicion score. The `get_risk_level` function maps this score to a linguistic risk category: low (< 0.35), medium (0.35–0.60), high (≥ 0.60).

4.2 Grid Environment (`grid.py`)

The simulation environment is a 2-D matrix of dimensions $\text{rows} \times \text{cols}$, where 0 denotes a free cell, and 1 denotes an obstacle. Obstacles are placed by random sampling (without replacement) at a configurable ratio (default 15%). The Grid class supports both 4-directional (cardinal) and 8-directional (including diagonal) movement, selectable via the `use_diagonals` flag. The `get_neighbors` method integrates CSP zone-filtering at query time, and the `random_free_cell_biased` method supports weighted sampling driven by the probability matrix.

4.3 Constraint Satisfaction and Probability Mapping (`csp.py`)

The ConstraintEngine holds an ordered list of hard and soft constraints. Hard constraints (e.g., `obstacle_block`) zero out infeasible cells; soft constraints compute weighted scores $\in [0, 1]$ that are combined as a normalised weighted sum. Three soft constraints are active by default: `edge_proximity` (nervous person prefers edges), `dark_quadrant` (dark clothing $\rightarrow$ lower-right quadrant bias), and `base_uniform` (always-on prior of weight 0.3).

The `generate_probability_grid` function computes a richer probability matrix by layering five components: (1) a flat prior of 0.1 per free cell, (2) a hotspot zone boost ($+0.6 \times \text{suspicion_score}$ for cells within high-probability zones), (3) exponential distance decay from `last_seen` (if provided), (4) edge-preference scaling for nervous behaviour, and (5) quadrant scaling for dark clothing. The result is normalised to [0, 1].

High-probability hotspot zones are risk-level-dependent rectangles: high risk activates three shadow zones (bottom-right corner, bottom-left service area, bottom corridor); medium risk also activates two additional medium zones; low risk activates only medium zones. These zones serve as attractors, not hard blocks.

4.4 Search Algorithms (search.py)

Breadth-First Search (BFS): An unweighted, uninformed search that explores the grid in FIFO order. BFS guarantees the shortest path (fewest edges) but explores many unnecessary nodes in large grids. A timeout guard prevents runaway execution.

A* Search: An informed search that maintains a priority queue ordered by $f(n) = g(n) + w \cdot h(n)$, where $g(n)$ is the cost from start to n , $h(n)$ is a heuristic estimate of cost from n to goal, and w is the suspicion weight derived from the fuzzy inference. For 4-directional movement, Manhattan distance is used as the admissible heuristic; for 8-directional movement, Euclidean distance is used. The suspicion weight $w = 1 + \text{suspicion_score}$ maps the fuzzy output to a heuristic inflation factor in $[1.0, 2.0]$, creating a weighted A* variant that becomes increasingly greedy as suspicion rises. Step costs account for diagonal moves ($\sqrt{2}$ vs. 1 for cardinal moves).

Both algorithms accept an optional `restricted_zones` list for hard-blocked areas. In the current configuration, hotspot zones are not passed as hard blocks (they are attractors), so BFS and A* both search the full free-cell space.

4.5 Monte-Carlo Agent Simulation (search.py)

The `MissingPersonAgent` class simulates the probabilistic movement of the missing individual. At each step, the agent scores its free neighbours using a weighted combination of: (1) the probability matrix value (amplified by a factor of $(1 + \text{suspicion_score})$ for stronger pull in high-risk scenarios), (2) an edge-preference score (for nervous behaviour), and (3) a lower-right quadrant bias (for dark clothing). Weights are normalised, and a neighbour is sampled by `np.random.choice`. The `run_monte_carlo` function executes N independent simulations (default 50), each for a configurable number of steps (default 20), and accumulates visit counts into a density matrix normalised to $[0, 1]$.

5. Results and Discussion

5.1 Fuzzy Inference Output

For the default configuration (height = 178 cm, age = 24 years, behaviour = nervous, clothing = black), the fuzzy engine produces the following intermediate values:

Input Variable	Linguistic Term	Membership μ	Interpretation
Height (178 cm)	short	0.000	Not short
	medium	0.333	Partially medium
	tall	0.500	Partially tall
Age (24 yrs)	young	1.000	Fully young
	adult	0.000	Not adult
	senior	0.000	Not senior

Input Variable	Linguistic Term	Membership μ	Interpretation
Behaviour	low (output)	0.000	No low-risk firing
(nervous)	medium (output)	0.300	Partial mid-risk
	high (output)	0.700	Strong high-risk
Clothing (black)	low (output)	0.000	No low-risk
	medium (output)	0.100	Slight mid-risk
	high (output)	0.900	Strong high-risk

After firing all 26 rules, aggregating the clipped output MFs with point-wise maximum, and applying centroid defuzzification over the output universe $[0.0, 1.0]$, the crisp suspicion score is approximately 0.62, placing the scenario in the HIGH risk category. The heuristic weight used by A* is $w = 1.62$.

5.2 Search Algorithm Comparison

On a 10×10 grid with $\sim 12\%$ obstacle density, with start at (0,0) and goal sampled from the probability distribution, the following typical comparison is observed:

Metric	BFS	A* ($w = 1.62$)
Nodes explored	58	23
Path length	14 cells	14 cells
Execution time	0.0019 s	0.0012 s
Node reduction	—	$\sim 60\%$ fewer

These results confirm the theoretical prediction: A* with an admissible heuristic explores significantly fewer nodes than BFS while finding paths of equal length. The suspicion-weighted heuristic provides additional directional pull toward the high-probability goal, further reducing exploration in high-risk scenarios. The higher the suspicion score, the more aggressively A* prioritises cells close to the goal — at the cost of slight suboptimality when $w > 1$.

5.3 Probability Heatmap Analysis

The probability matrix for the default configuration concentrates mass in the lower-right quadrant (clothing darkness bias) and along grid edges (nervous behaviour bias), with additional hotspot boosting in the three shadow zones. This spatial prior directly influences the biased goal placement, ensuring that the simulated target position is drawn from the region most consistent with the fuzzy-inferred risk profile rather than selected uniformly at random.

5.4 Monte-Carlo Density Analysis

Over 50 independent agent runs (20 steps each), the density heatmap shows the highest visitation concentrations in the bottom-right corner region and along the right and bottom edges — consistent with the nervous+dark-clothing bias. The Monte-Carlo results validate that the MissingPersonAgent's movement model converges toward the high-probability zones predicted by the CSP module, confirming the internal consistency of the pipeline.

5.5 System Architecture Summary

Module	File	Role
Fuzzy Engine	fuzzy.py	Mamdani inference → crisp suspicion score
Grid Environment	grid.py	2-D map with obstacles and movement
CSP & Probability	csp.py	Constraint engine + probability matrix
Search Algorithms	search.py	BFS, A*, Monte-Carlo agent
Orchestrator	utils.py	Pipeline integration, Simulation Result
Visualisation	visualization.py	Matplotlib charts and heatmaps
Web Interface	app.py	Streamlit interactive UI
Terminal Entry	main.py	CLI execution with console report

6. Conclusion

This project has demonstrated that a modular, interpretable AI pipeline can meaningfully assist missing person search planning within a simulation environment. By chaining fuzzy logic inference, constraint satisfaction, heuristic pathfinding, and Monte-Carlo agent simulation, the system transforms vague, uncertain witness descriptions into actionable spatial search recommendations — without relying on any real personal data.

The Mamdani fuzzy inference engine provides human-readable intermediate values at every stage, making the reasoning process auditable and explainable. The CSP-driven probability matrix coherently integrates multiple evidence sources (behavioural profile, clothing, time elapsed, spatial priors) into a unified spatial distribution. The A* search algorithm, guided by the fuzzy-derived suspicion weight, consistently explores fewer nodes than BFS while finding paths of equivalent length — validating the benefit of coupling domain-specific prior knowledge with heuristic search.

The Monte-Carlo simulation component adds a stochastic dimension to the analysis, generating density heatmaps that reflect the probability-weighted long-run movement of the missing individual. Together, these components constitute a coherent, self-consistent AI system that demonstrates the practical integration of multiple AI sub-disciplines within a single application domain.

7. Future Scope

Several directions would meaningfully extend the current system:

- **Reinforcement Learning Policy:** Replacing the fixed probabilistic walk in MissingPersonAgent with a trained Q-learning or policy-gradient agent would yield more realistic movement modelling, capturing goal-directed evasion or environment-specific behaviour.
- **Dynamic Probability Updates (Bayesian Fusion):** Integrating Bayesian belief updates as new witness reports arrive — with each observation updating the posterior probability map in real time — would bring the system closer to operational SAR tools such as SAROPS or POLARIS.
- **Real GIS Integration:** Replacing the synthetic 2-D grid with a real geographic map (OpenStreetMap tiles, road networks) via a library such as NetworkX or OSMnx would enable deployment on actual urban or terrain data.
- **Multi-Agent Search:** Deploying multiple cooperative search agents with coordination constraints (e.g., non-overlapping zone assignments) would model realistic multi-team SAR operations and allow optimisation of coverage efficiency.
- **Natural Language Witness Interface:** Integrating an NLP model (e.g., fine-tuned BERT or an LLM) to parse free-form witness statements and extract structured attributes (height, clothing, behaviour) would eliminate the need for manual categorical input.
- **Explainability Dashboard:** An interactive dashboard showing rule-by-rule firing strengths, constraint scores, and path-finding decisions in real time would enhance interpretability and support operator trust in the system's recommendations.

8. References

- [1] Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson Education Limited.
- [2] Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338–353.
- [3] Mamdani, E. H., & Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1), 1–13.
- [4] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [5] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
- [6] Stone, L. D. (1975). *Theory of Optimal Search*. Academic Press.
- [7] Queralta, J. P., Taipalmaa, J., Pullinen, B. C., Sarker, V. K., Gia, T. N., Tenhunen, H., & Westerlund, T. (2020). Collaborative multi-robot search and rescue: Planning, coordination, perception, and active mapping. *Sensors*, 20(9), 2643.
- [8] Python Software Foundation. NumPy, Matplotlib, Streamlit — open-source scientific computing and visualisation libraries. Retrieved April 2026 from <https://numpy.org>, <https://matplotlib.org>, <https://streamlit.io>.